

DECLARATION

I understand that this is an individual assessment and that collaboration is not permitted. I have not received any assistance with my work for this assessment. Where I have used the published work of others, I have indicated this with appropriate citation.

I have not and will not share any part of my work on this assessment, directly or indirectly, with any other student.

I have read and understood the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

I have also read and understood the guide, and completed the 'Ready Steady Write' Tutorial on avoiding plagiarism, located at <https://libguides.tcd.ie/academic-integrity/ready-steady-write>.

I understand that by returning this declaration with my work, I am agreeing with the above statement.

Name: Stephen Komolafe

Date: 02/01/2026

Final Assignment 2025-26

Part 1 - Math GPT:

1.1. Dataset Construction

To study whether a character-level transformer can learn symbolic arithmetic, a Python script, `make_math_data.py` [A.2], was written to construct several synthetic arithmetic datasets. All expressions are deterministic and generated within their respective domains, resulting in the complete coverage of the input space and a single unambiguous numerical result for each. This allows for accurate correctness evaluation while avoiding biases introduced by random sampling.

Each expression is represented as a character sequence of the form $a \text{ (op) } b = c$, where a and b are operands (non-negative integers), op denotes the arithmetic operator, and c is the exact result of the operation. The equals sign ($=$) serves as a clear delimiter between the input expression and the target output, allowing for autoregressive generation of the answer.

➤ Addition Dataset

The addition dataset contains expressions of the form $a + b = c$, where operands range from 0 to 99. All possible operand pairs within this range are included, resulting in an exhaustive dataset that requires the model to learn digit-wise addition and carry propagation. This dataset serves as the primary benchmark for evaluating the arithmetic reasoning of the model and is used exclusively in the evaluation of the baseline model (see 1.2) and an optimised model (see 1.3).

The datasets listed below were created solely to assess the arithmetic capabilities of the optimised model, as described in task 1.4. These datasets assess the model's ability to generalise beyond addition and whether architectural adaptations learnt for addition apply to other arithmetic operations.

➤ Subtraction Dataset

The subtraction dataset consists of expressions of the form $a - b = c$, with operands constrained to $a \geq b$, for non-negative results. This restriction removes ambiguity around negative numbers while still requiring borrow operations. The dataset contains all valid operands.

➤ Multiplication Dataset

The multiplication dataset contains expressions of the form $a \times b = c$, with operands again ranging from 0 to 99. Compared to addition and subtraction, multiplication produces a wider range of output magnitudes and introduces more complex digit interactions, providing a more rigorous test of the model's ability to learn algorithmic structure.

➤ Division Dataset

The division dataset includes expressions of the form $a \div b = c$, restricted to cases where the quotient c is an integer. These constraints avoid fractional outputs and guarantee a single correct result for each expression. While division is a computationally complex operation in general, the constrained dataset still tests the model's ability to represent inverse arithmetic relationships.

➤ Combined Arithmetic Dataset

A final dataset combines all four arithmetic operations (addition, subtraction, multiplication, and division) into a single corpus. This dataset was constructed to determine whether the model can learn and distinguish multiple arithmetic rules simultaneously, or whether interference between operations degrades performance.

All datasets were randomly shuffled and written as plain text files with one expression per line in the form [expression=result]. Their exhaustive construction ensures complete coverage of the defined expression spaces while remaining compact and computationally tractable.

1.2 Evaluation Metrics

Arithmetic reasoning requires precise accuracy, motivating evaluation measures that go beyond minimising training loss. All evaluation metrics in this section are first applied to a baseline GPT model that retains the original architecture from the reference implementation [A.1]. This baseline serves as a comparison point for the architectural changes described in Section 1.3.

Evaluation is performed by prompting the model with the left-hand side of each arithmetic expression (e.g., $35+99=$) and generating the result autoregressively. The generated output is then compared directly against the ground-truth value. During training, cross-entropy loss is monitored to assess optimisation stability and convergence.

To evaluate functional correctness, exact-match accuracy is used as the primary metric. This measures the proportion of test expressions for which the predicted output exactly matches the correct result. Even minor numerical errors invalidate an arithmetic answer, making exact-match accuracy a strict but appropriate measure of performance.

To provide additional insight into partial correctness, token-level accuracy is also reported. This metric measures the fraction of output characters that match the ground truth and captures cases where the model produces outputs of the correct length or approximate magnitude but fails to compute the precise result.

Baseline Model (10.75M parameters)		
Exact-match accuracy	0.0370	
Token-level accuracy	0.3819	
Training loss @ 999 iters	1.3025	
Validation loss @ 999 iters	1.3029	
Sample predictions		
35+99=147 (target: 134)	79+67=117 (target: 146)	97+24=115 (target: 121)
63+64=112 (target: 127)	57+98=150 (target: 155)	34+95=136 (target: 129)

After 1,000 training iterations, the baseline model achieves a training loss of 1.3025 and a validation loss of 1.3029, indicating stable convergence without significant overfitting. However, loss alone is insufficient for assessing arithmetic competence: even with stable optimisation, the model achieves an exact-match accuracy of only 3.7%, indicating that it rarely computes correct sums.

Token-level accuracy was significantly higher at 38.2%, indicating that the model has learnt aspects of output formatting and local digit structure. However, the sizable difference between token-level and exact-match accuracy suggests failure to consistently apply the addition algorithm. Qualitative inspection of sample predictions confirms this pattern: the model frequently generates numerically plausible outputs with the correct number of digits, but introduces systematic errors in carry propagation, indicating a lack of true algorithmic understanding.

1.3 Architectural Adaptations and Training Schedule Optimisation

Despite stable optimisation behaviour, the baseline model displayed extremely low arithmetic accuracy, indicating a mismatch between model capacity and task structure. Symbolic arithmetic is deterministic, low-dimensional, and highly structured, implying that increasing model size is unnecessary and potentially harmful. Instead, measures to simplify the model's architecture were investigated to

encourage the model to learn consistent, rule-based transformations rather than memorising spurious patterns.

To this end, the model architecture was downsized. The embedding dimension was reduced from 384 to 64, the number of attention heads from 6 to 4, and the number of transformer layers from 6 to 2. Together, these changes reduced the parameter count from 10.75 million to 0.103 million, representing a reduction of approximately two orders of magnitude. This reduction constrains model capacity, encouraging the model to concentrate on learning the fundamental structure of the arithmetic task rather than overfitting to surface-level token correlations.

The dataset's arithmetic expressions are short and fixed in structure, so long-range context is unnecessary and potentially distracting for the model. Dropout was also reduced from 0.2 to 0.1, as arithmetic computation benefits from precise internal representations rather than stochastic regularisation, which can introduce noise into intermediate digit-level reasoning.

The smaller architecture facilitated a longer training period without increasing computational cost. The number of training iterations was increased from 1,000 to 5,000, allowing the model to converge more fully. Training and validation losses decreased smoothly throughout training, eventually converging to approximately 1.20, indicating stable optimisation with no evidence of overfitting.

These architectural and training adaptations resulted in a substantial improvement in arithmetic performance. Exact-match accuracy increased from 3.7% in the baseline model to 36.4%, while token-level accuracy improved from 38.2% to 65.6%. This demonstrates that the optimised architecture is better suited to the structure of the arithmetic task, even with significantly fewer parameters.

Optimised Model (0.103M parameters)		
Exact-match accuracy	0.3640	
Token-level accuracy	0.6555	
Training loss @ 4999 iters	1.2017	
Validation loss @ 4999 iters	1.1986	
Sample predictions		
3+6=13 (target: 9)	72+48=120 (target: 120)	29+11=40 (target: 40)
16+7=114 (target: 23)	88+57=154 (target: 145)	86+52=138 (target: 138)

Qualitative inspection of the sample predictions confirms this quantitative improvement. Several two-digit additions that were consistently incorrect under the baseline model are now computed correctly. However, remaining errors are primarily caused by inconsistent carry propagation in multi-digit sums, indicating that while the model has partially internalised the addition procedure, it has yet to learn a fully robust algorithmic solution.

1.4 Learning Different Arithmetic Operations

To assess the model's ability to learn different forms of symbolic arithmetic, the optimised architecture was trained separately on addition, subtraction, multiplication, division datasets, and a combined dataset containing all operations. Evaluation uses the metrics defined in Section 1.2. Although training and validation loss were monitored, accuracy-based metrics were prioritised, as loss does not directly reflect semantic correctness in arithmetic tasks.

Addition

Under the optimised architecture, addition performance improves substantially relative to earlier experiments. After extended training, the model achieved an exact-match accuracy of 71.9% and a token-level accuracy of 84.8%, indicating that most sums are now computed correctly. These results demonstrate that between training the model immediately after optimisations were made and then training it again to ascertain the results of learning the addition dataset, the model has largely learnt digit-

wise addition and carry propagation, correctly handling many multi-digit cases that previously failed. Token-level accuracy remains higher than exact-match accuracy, reflecting occasional single-digit errors that invalidate otherwise plausible outputs.

Sample Predictions		
90+75=165 (target: 165)	36+58=94 (target: 94)	83+7=71 (target: 90)
27+5=107 (target: 32)	38+79=117 (target: 117)	65+9=74 (target: 74)

It is evident that remaining failures are dominated by inconsistent carry handling in less frequent digit configurations (e.g., $83+7=71$) and rare digit misalignment errors (e.g., $27+5=107$). While the model has not learnt a perfectly robust addition algorithm, the substantial improvement relative to earlier runs indicates that addition is now mostly learnt under the optimised configuration, rather than merely approximated.

Subtraction

Subtraction is learnt more successfully, achieving 74.1% exact-match accuracy and 84.9% token-level accuracy, indicating reliable performance across most test cases.

Sample Predictions		
40-37=3 (target: 3)	41-30=11 (target: 11)	72-23=49 (target: 49)
14-1=23 (target: 13)	73-41=32 (target: 32)	64-13=51 (target: 51)

Errors are infrequent and typically involve isolated borrowing mistakes. This strong performance can be attributed to the dataset design, restricting subtraction to non-negative results, reducing the complexity of borrow propagation relative to addition.

Multiplication

Multiplication yields mixed results. The model achieved 68.5% exact-match accuracy and 79.8% token-level accuracy but exhibited signs of overfitting.

Sample Predictions		
14*16=224 (target: 224)	10*7=70 (target: 70)	19*11=199 (target: 209)
2*10=20 (target: 20)	9*8=8 (target: 72)	13*13=169 (target: 169)

Errors such as $9*8=8$ and $19*11=199$ suggest that the model appears to memorise frequent multiplication trends but lacks a consistent algorithmic understanding of multiplication.

Division

Division was learnt exceptionally well under the constrained dataset conditions. The model achieved the highest performance among all individual tasks, with an exact-match accuracy of 83.7% and a token-level accuracy of 89.5%.

Sample Predictions		
10/2=5 (target: 5)	75/75=1 (target: 1)	51/1=51 (target: 51)
56/1=56 (target: 56)	8/4=22 (target: 2)	25/1=55 (target: 25)

Errors tend to involve incorrect digit repetition or spurious outputs rather than structural misunderstanding. Strong performance is largely attributable to dataset constraints that include only exact integer quotients. Many examples reduce to identity-like mappings (e.g., division by 1 or by the number itself), making the task easier to memorise than unconstrained division.

Combined Operations

When trained on the balanced dataset containing all four operations, performance degraded substantially across all metrics. The model achieved an exact-match accuracy of 6.9% and a token-level accuracy of 32.1%, implying severe difficulty in handling multiple operations simultaneously.

Sample Predictions		
40/2=30 (target: 20)	88+60=133 (target: 148)	83/83=1 (target: 1)
75-39=32 (target: 36)	5+53=121 (target: 58)	88-82=0 (target: 6)

The model frequently applies incorrect arithmetic rules, showing that it fails to learn robust operator-specific representations when trained jointly on all tasks.

Part 2 - Boolean Logic GPT:

While Boolean logic operates on a small and discrete state space, it presents challenges distinct from arithmetic, such as including operator semantics, unary negation, and compositional reasoning enforced through parentheses. Thus, the aim of this section is to assess whether the same GPT architecture used previously can internalise these logical rules solely through sequence modelling, as well as how performance changes as structural complexity increases.

2.1 Dataset Construction

To investigate increasing levels of logical complexity, a second Python script, `make_bool_data.py` [A.3], was written to construct three Boolean logic datasets. All expressions are deterministic and exhaustively generated within their respective domains, ensuring that each dataset encompasses the entire expression space. Each expression is associated with a single correct Boolean outcome, allowing for a clear evaluation.

➤ Simple Boolean Dataset

This dataset consists exclusively of flat Boolean expressions with no nesting. Unary expressions apply logical negation (*NOT*) to Boolean literals (*True*, *False*), whereas binary expressions include all combinations of the operators *AND*, *OR*, and *XOR* applied to pairs of Boolean values. The purpose of this dataset is to test whether the model can learn the semantics of individual Boolean operators.

➤ Nested Boolean Dataset

To evaluate compositional reasoning, a second dataset was generated containing nested expressions of the form $[(A \text{ } op_1 \text{ } B) \text{ } op_2 \text{ } C]$, where *A*, *B*, and *C* are Boolean literals and *op*₁, *op*₂ are binary operators. Parentheses explicitly specify evaluation order, requiring the model to compute an intermediate logical result before applying a second operator. This dataset tests the model's ability to represent and combine intermediate states without relying solely on local token patterns.

➤ Combined Boolean Dataset

The final dataset merges the simple and nested datasets into a single training corpus. This tests whether the model can simultaneously learn unary, binary, and nested Boolean logic without interference, along with whether exposure to simpler expressions aids generalisation on more complex ones.

All datasets were randomly shuffled and written as plain text files with one expression per line in the form `[expression = result]`. Due to their exhaustive construction, these datasets avoid sampling bias while remaining compact.

2.2 Evaluation Metrics

Boolean reasoning allows a single correct output for each input expression, which motivates evaluation metrics that prioritise exact correctness over probabilistic similarity. As in Part 1, evaluation is first conducted on a baseline GPT model built on the original reference architecture [A.1], which serves as a comparison point for the architectural adaptations explored in Section 2.3.

For each test case, the model is prompted with the Boolean expression up to the equals sign (e.g. *True AND False* =) and generates the result autoregressively. The generated output is then compared directly against the ground-truth Boolean value.

The primary evaluation metric is exact-match accuracy, which measures the proportion of expressions for which the predicted output precisely matches the correct Boolean result (*True* or *False*). This metric is particularly appropriate for Boolean logic, as even small deviations correspond to logically incorrect evaluations.

Token-level accuracy assesses character-level agreement between the predicted output and the ground truth. Although less strict, this metric shed light into partial correctness and aids in distinguishing reasoning errors from formatting or termination issues.

Training and validation cross-entropy loss were monitored throughout training to assess optimisation stability and overfitting behaviour. However, as with arithmetic reasoning, loss values alone do not reliably reflect semantic correctness and thus cannot be used as primary performance indicators.

Both the baseline and optimised models evaluated in this section were trained on the simple Boolean dataset described in Section 2.1, so that performance differences reflect architectural choices rather than dataset complexity.

Baseline Model (10.66M parameters)		
Exact-match accuracy	0.7857	
Token-level accuracy	0.8065	
Training loss @ 999 iters	0.1680	
Validation loss @ 999 iters	0.7893	
Sample predictions		
True AND True=True (target: True)	True AND False=False (target: False)	False AND False=False (target: False)
False XOR False=True (target: False)	True OR False=False (target: True)	NOT True=False (target: False)

After 1,000 training iterations, the baseline model performs well, with an exact-match accuracy of 78.6% and a token-level accuracy of 80.7%. Training loss decreases rapidly to 0.1680, while validation loss remains higher at 0.7893, indicating a degree of overfitting but no corresponding collapse in logical accuracy.

A qualitative inspection of sample predictions reveals systematic failure modes. While many predictions are correct, the model occasionally produces logically inconsistent outputs, such as incorrect XOR or OR evaluations. These errors indicate that, despite its high overall accuracy, the baseline model does not fully internalise all Boolean operator semantics and instead relies partially on pattern frequency rather than strict logical rules.

2.3 Architectural Adaptations

To ensure a controlled comparison, the same downsized GPT architecture introduced in Part 1 was reused, thus featuring the same 64-dimensional embedding, 4 attention heads, 0.1 dropout, and 2 transformer layer configuration.

Boolean expressions are significantly shorter than arithmetic sequences and using the original context length caused invalid batch sampling on the smaller datasets. To remedy this, the model's context window (`block_size`) was reduced from 32 to 16 tokens to better reflect the structure of Boolean inputs. This adjustment yields around 0.103 million parameters, depending on the dataset vocabulary.

Reducing the context length improves training stability while preserving expressive capacity, as all Boolean expressions comfortably fit within the smaller window. Boolean evaluation naturally aligns with next-character prediction, so no changes to the loss function or training procedure were required.

Optimised Model (0.103M parameters)		
Exact-match accuracy	0.9286	
Token-level accuracy	0.9355	
Training loss @ 4999 iters	0.1649	
Validation loss @ 4999 iters	0.9776	
Sample predictions		
True AND True=True (target: True)	NOT False=True (target: True)	True AND False=False (target: False)
True OR False=True (target: True)	False XOR True=True (target: True)	False OR False=False (target: False)

Already, the optimised model achieves significantly higher accuracy than the baseline, despite its much smaller size. Although validation loss increases with extended training, logical accuracy remains high, indicating overfitting in probability space without compromising functional correctness.

2.4 Learning Boolean Operations

The optimised model was evaluated across Boolean datasets of increasing complexity to assess its ability to generalise beyond flat expressions.

Simple Boolean Expressions

The model performs exceptionally well with simple Boolean logic. When trained on the simple dataset, it achieved an exact-match accuracy of 92.9% and a token-level accuracy of 93.6%. Sample predictions demonstrate consistently correct evaluation across all operators, including XOR and unary negation:

Sample predictions	
False OR True=True (target: True)	False OR False=False (target: False)
True AND False=False (target: False)	True XOR True=False (target: False)
True OR False=True (target: True)	NOT False=True (target: True)

Although validation loss increases later in training, accuracy remains high, indicating mild overfitting without loss of functional correctness. These results demonstrate that the transformer can reliably learn basic Boolean operator semantics when expressions are flat and local.

Nested Boolean Expressions

Nested expressions significantly reduce performance. The model achieved an exact-match accuracy of 66.7% and a token-level accuracy of 73.4%. While many expressions are evaluated correctly, errors frequently occur when intermediate results must be computed and reused:

Sample predictions	
(False OR False) AND True=False (target: False)	(False OR False) XOR True=False (target: True)
(False XOR True) OR True=True (target: True)	(True XOR True) OR True=True (target: True)
(False OR True) XOR True=True (target: False)	(False AND False) AND True=True (target: False)

These failures suggest that it is difficult to maintain and combine intermediate Boolean states across parentheses. Despite correct handling of individual operators, the model struggles to consistently propagate intermediate results, indicating that attention alone may not be enough for reliable multi-step logical composition.

Combined Boolean Expressions

Training on the combined dataset yields a 72.1% exact-match accuracy and 77.7% token-level accuracy, outperforming the nested-only model while falling short of the simple-only case.

Sample predictions	
False AND False=False (target: False)	(True OR False) AND False=False (target: False)
(False XOR True) AND True=True (target: True)	True XOR False=True (target: True)
(True XOR False) AND True=False (target: True)	(True AND True) XOR False=True (target: True)

Sample predictions show strong performance on simple expressions and moderate success with nested ones. Joint training appears to have a modest regularising effect, improving nested performance over training on nested expressions alone. However, consistent errors remain in XOR-heavy and deeply compositional cases.

Part 3 - Discussion:

The Math GPT and Boolean GPT experiments demonstrate how transformer architectures react differently to symbolic tasks with distinct structural requirements. While both arithmetic and Boolean reasoning are deterministic with a single correct output, their algorithmic complexity varies significantly, resulting in different learning outcomes.

One important shared finding is that architectural downsizing consistently improved functional accuracy. In both tasks, the large baseline model ($\approx 10\text{M}$ parameters) achieved low training loss but poor semantic correctness, suggesting overfitting to token patterns rather than rule-based reasoning. Despite a two-order-of-magnitude reduction in parameters, reducing embedding size, number of layers, attention heads, and context length resulted in gains in exact-match accuracy. This demonstrates that symbolic reasoning benefits from capacity constraints that promote rule learning over memorisation.

However, the two tasks diverge in terms of difficulty in reasoning. Boolean reasoning was learnt consistently, with the optimised model achieving over 92% exact-match accuracy on simple expressions. Boolean operators act over a small, discrete state space and can be captured through local transformations, making them ideal for attention-based sequence models. Errors mainly arose in nested expressions, where the model struggled to propagate intermediate logical results, revealing limitations in compositional depth rather than operator understanding.

Arithmetic reasoning proved more challenging, although not always unsuccessful. Under the optimised architecture and extended training, addition performance improved substantially, with exact-match accuracy exceeding 70%. This demonstrates how the model can partially internalise digit-wise addition and carry propagation. However, persistent errors in rarer carry configurations and in combined-operation settings suggest that arithmetic algorithms are learnt incompletely rather than as fully general procedures. Standard transformers continue to struggle with tasks that require the consistent reuse of intermediate numerical states.

These disparities are highlighted even further by evaluation metrics. In both tasks, cross-entropy loss correlated poorly with semantic correctness, emphasising the need for exact-match accuracy in symbolic domains. Token-level accuracy proved useful for diagnostic purposes but insufficient as a primary metric, as small local errors invalidated entire arithmetic results.

The rigorous training and testing of the transformer on an assortment of arithmetic- and Boolean-logic-centric datasets revealed that transformers can learn symbolic rules when they are shallow and locally compositional but struggle with tasks requiring robust, multi-step algorithmic state. While architectural simplification and longer training had helped in mitigating some limitations, fundamental challenges in intermediate state representation remain, particularly for carry propagation during arithmetic reasoning.

Appendix

Appendix A - Code

A.1 – [gpt.py]

```
import torch

import torch.nn as nn

from torch.nn import functional as F

import random

import os

from make_math_data import generate_math_datasets

from make_bool_data import generate_boolean_datasets


# hyperparameters

batch_size = 64 # how many independent sequences will we process in parallel?

#block_size = 256 # what is the maximum context length for predictions?

max_iters = 5000

eval_interval = 500

learning_rate = 3e-4

device = 'cuda' if torch.cuda.is_available() else 'cpu'

eval_iters = 200

'''#original

n_embd = 384

n_head = 6

n_layer = 6

...

#Wk9 Assignment config

n_embd = 64

n_head = 4

n_layer = 2

dropout = 0.1

# -----

torch.manual_seed(1337)

datasets = { #-----math-----#

    "mADD": "input_addition.txt",

    "mSUB": "input_subtraction.txt",

    "mMUL": "input_multiplication.txt",
```

```

        "mDIV": "input_division.txt",
        "mALL": "input_all_ops.txt",
        #-----boolean-----#
        "bSIMP": "input_bool_simple.txt",
        "bNEST": "input_bool_nested.txt",
        "bALL": "input_bool_all.txt"
    }

#-----
chosen = "mALL"
#-----

if chosen.startswith("b"):
    block_size = 16 # for bool dataset
else:
    block_size = 32 # for math dataset

dataset_path = datasets[chosen]

if not os.path.exists(dataset_path):
    print(f"Dataset '{dataset_path}' missing. Generating datasets...")

    if chosen.startswith("m"):
        generate_math_datasets()
    elif chosen.startswith("b"):
        generate_boolean_datasets()

    if not os.path.exists(dataset_path):
        raise FileNotFoundError(
            f"Dataset generation failed - '{dataset_path}' still missing."
        )

with open(dataset_path, 'r', encoding='utf-8') as f:
    text = f.read()

# wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
# with open(datasets[chosen], 'r', encoding='utf-8') as f:
#     text = f.read()

```

```

# here are all the unique characters that occur in this text
chars = sorted(list(set(text)))
vocab_size = len(chars)

# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }

encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a string


# Train and test splits
data = torch.tensor(encode(text), dtype=torch.long)

n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]


# evaluation functions

def generate_answer(model, prompt, max_new_tokens=6):
    model.eval()

    idx = torch.tensor([encode(prompt)], dtype=torch.long).to(device)

    with torch.no_grad():
        out = model.generate(idx, max_new_tokens=max_new_tokens)

    generated = decode(out[0].tolist())

    # return only the generated answer (strip prompt and newline)
    return generated[len(prompt):].split("\n")[0]


def load_test_expressions(filename, max_samples=1000):
    with open(filename, "r") as f:
        data = [line.strip() for line in f]

    random.shuffle(data)

    return data[:max_samples]


def exact_match_accuracy(model, test_data):
    correct = 0

    for expr in test_data:
        lhs, rhs = expr.split("=")

        prompt = lhs + "="

        prediction = generate_answer(model, prompt)

```

```

        if prediction == rhs:
            correct += 1
    return correct / len(test_data)

def token_level_accuracy(model, test_data):
    correct_tokens = 0
    total_tokens = 0
    for expr in test_data:
        lhs, rhs = expr.split("=")
        prompt = lhs + "="
        prediction = generate_answer(model, prompt)
        for p, t in zip(prediction, rhs):
            if p == t:
                correct_tokens += 1
            total_tokens += 1
    return correct_tokens / total_tokens

def print_sample_predictions(model, test_data, n=10):
    print("\nSample predictions:")
    for expr in random.sample(test_data, n):
        lhs, rhs = expr.split("=")
        prompt = lhs + "="
        prediction = generate_answer(model, prompt)
        print(f"{prompt}{prediction} (target: {rhs})")

# data loading
def get_batch(split):
    # generate a small batch of data of inputs x and targets y
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y

@torch.no_grad()
def estimate_loss():
    out = {}

```

```

model.eval()

for split in ['train', 'val']:
    losses = torch.zeros(eval_iters)
    for k in range(eval_iters):
        X, Y = get_batch(split)
        logits, loss = model(X, Y)
        losses[k] = loss.item()
    out[split] = losses.mean()

model.train()

return out

```

```

class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # input of size (batch, time-step, channels)
        # output of size (batch, time-step, head size)
        B, T, C = x.shape
        k = self.key(x) # (B,T,hs)
        q = self.query(x) # (B,T,hs)
        # compute attention scores ("affinities")
        wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5 # (B, T, hs) @ (B, hs, T) -> (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
        wei = F.softmax(wei, dim=-1) # (B, T, T)
        wei = self.dropout(wei)
        # perform the weighted aggregation of the values
        v = self.value(x) # (B,T,hs)
        out = wei @ v # (B, T, T) @ (B, T, hs) -> (B, T, hs)
        return out

```

```

class MultiHeadAttention(nn.Module):

```

```

""" multiple heads of self-attention in parallel """

def __init__(self, num_heads, head_size):
    super().__init__()

    self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])

    self.proj = nn.Linear(head_size * num_heads, n_embd)

    self.dropout = nn.Dropout(dropout)

def forward(self, x):
    out = torch.cat([h(x) for h in self.heads], dim=-1)

    out = self.dropout(self.proj(out))

    return out

class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

def __init__(self, n_embd):
    super().__init__()

    self.net = nn.Sequential(
        nn.Linear(n_embd, 4 * n_embd),
        nn.ReLU(),
        nn.Linear(4 * n_embd, n_embd),
        nn.Dropout(dropout),
    )

def forward(self, x):
    return self.net(x)

class Block(nn.Module):
    """ Transformer block: communication followed by computation """

def __init__(self, n_embd, n_head):
    # n_embd: embedding dimension, n_head: the number of heads we'd like
    super().__init__()

    head_size = n_embd // n_head

    self.sa = MultiHeadAttention(n_head, head_size)

    self.ffwd = FeedFoward(n_embd)

    self.ln1 = nn.LayerNorm(n_embd)

    self.ln2 = nn.LayerNorm(n_embd)

```

```

def forward(self, x):
    x = x + self.sa(self.ln1(x))
    x = x + self.ffwd(self.ln2(x))
    return x

class GPTLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

        # better init, not covered in the original GPT video, but important, will cover in followup video
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx and targets are both (B,T) tensor of integers
        tok_emb = self.token_embedding_table(idx) # (B,T,C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
        x = tok_emb + pos_emb # (B,T,C)
        x = self.blocks(x) # (B,T,C)
        x = self.ln_f(x) # (B,T,C)
        logits = self.lm_head(x) # (B,T,vocab_size)

        if targets is None:
            loss = None

```



```

else:
    B, T, C = logits.shape
    logits = logits.view(B*T, C)
    targets = targets.view(B*T)
    loss = F.cross_entropy(logits, targets)

return logits, loss

def generate(self, idx, max_new_tokens):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]

        # get the predictions
        logits, loss = self(idx_cond)

        # focus only on the last time step
        logits = logits[:, -1, :] # becomes (B, C)

        # apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)

        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)

        # append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)

    return idx

model = GPTLanguageModel()
m = model.to(device)

# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for iter in range(max_iters):

    # every once in a while evaluate the loss on train and val sets
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()

        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

```

```

# sample a batch of data
xb, yb = get_batch('train')

# evaluate the loss
logits, loss = model(xb, yb)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()

# generate from the model
#context = torch.zeros([1, 1], dtype=torch.long, device=device)
#print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))

test_data = load_test_expressions(datasets[chosen], max_samples=1000)
exact_acc = exact_match_accuracy(model, test_data)
token_acc = token_level_accuracy(model, test_data)
print(f"\nExact-match accuracy: {exact_acc:.4f}")
print(f"Token-level accuracy: {token_acc:.4f}")
print_sample_predictions(model, test_data, n=10)
#open('more.txt', 'w').write(decode(m.generate(context, max_new_tokens=10000)[0].tolist()))

```

A.2 – [make_math_data.py]

```
import random

def generate_addition_data(max_operand):
    samples = []
    for a in range(max_operand + 1):
        for b in range(max_operand + 1):
            samples.append(f"{a}+{b}={a+b}")
    return samples

def generate_subtraction_data(max_operand):
    samples = []
    for a in range(max_operand + 1):
        for b in range(a + 1): #non-negative answers only (a - b >= 0)
            samples.append(f"{a}-{b}={a-b}")
    return samples

def generate_multiplication_data(max_operand):
    samples = []
    for a in range(max_operand + 1):
        for b in range(max_operand + 1):
            samples.append(f"{a}*{b}={a*b}")
    return samples

def generate_division_data(max_operand):
    samples = []
    for b in range(1, max_operand + 1):
        for a in range(b, max_operand + 1):
            if a % b == 0: #exact integer division only
                samples.append(f"{a}/{b}={a//b}")
    return samples

def write_dataset(filename, data):
    random.shuffle(data)
    with open(filename, "w") as f:
        for line in data:
            f.write(line + "\n")
    print(f"{filename}: {len(data)} samples")

#equal amount of samples from each operation (for combined dataset)
```

```

def balance_dataset(datasets, samples_per_op):
    balanced = []
    for data in datasets:
        random.shuffle(data)
        balanced.extend(data[:samples_per_op])
    return balanced

def generate_math_datasets():
    # addition dataset
    addition_data = (
        generate_addition_data(max_operand=9) +
        generate_addition_data(max_operand=99)
    )
    write_dataset("input_addition.txt", addition_data)

    # subtraction dataset
    subtraction_data = generate_subtraction_data(max_operand=99)
    write_dataset("input_subtraction.txt", subtraction_data)

    # multiplication dataset
    multiplication_data = generate_multiplication_data(max_operand=19)
    write_dataset("input_multiplication.txt", multiplication_data)

    # division dataset
    division_data = generate_division_data(max_operand=99)
    write_dataset("input_division.txt", division_data)

    # all operations combined
    all_ops = balance_dataset(
        [addition_data, subtraction_data, multiplication_data, division_data],
        samples_per_op=2000
    )
    write_dataset("input_all_ops.txt", all_ops)

if __name__ == "__main__":
    generate_math_datasets()

```

A.3-[make_bool_data.py]

```
import random
import itertools

BOOLS = ["True", "False"]
BIN_OPS = ["AND", "OR", "XOR"]

def eval_expr(expr):
    #remove parentheses + split each token in expression
    tokens = expr.replace("(", "").replace(")", "").split()
    #convert string to boolean
    def to_bool(x):
        return x == "True"
    # if single NOT operation
    if tokens[0] == "NOT":
        return not to_bool(tokens[1]) #move past NOT and evaluate the boolean value

    values = []
    ops = []

    #parse tokens
    for t in tokens:
        if t in {"True", "False"}:
            values.append(to_bool(t))
        elif t in {"AND", "OR", "XOR"}:
            ops.append(t)

    result = values[0]
    #evaluate binary operations left to right
    for i, op in enumerate(ops):
        if op == "AND":
            result = result and values[i + 1]
        elif op == "OR":
            result = result or values[i + 1]
        elif op == "XOR":
            result = (result != values[i + 1])

    return result

def generate_unary():
```

```

    samples = []

    for b in BOOLS:
        expr = f"NOT {b}"
        samples.append(f"{expr}={eval_expr(expr)}")

    return samples

def generate_binary():
    samples = []

    for a, b, op in itertools.product(BOOLS, BOOLS, BIN_OPS):
        expr = f"{a} {op} {b}"
        samples.append(f"{expr}={eval_expr(expr)}")

    return samples

def generate_nested():
    samples = []

    for a, b, c, op1, op2 in itertools.product(BOOLS, BOOLS, BOOLS, BIN_OPS, BIN_OPS):
        expr = f"({a} {op1} {b}) {op2} {c}"
        samples.append(f"{expr}={eval_expr(expr)}")

    return samples

def write_dataset(filename, data):
    random.shuffle(data)

    with open(filename, "w") as f:
        for line in data:
            f.write(line + "\n")

    print(f"{filename}: {len(data)} samples")

def generate_boolean_datasets():
    simple = generate_unary() + generate_binary()
    nested = generate_nested()

    write_dataset("input_bool_simple.txt", simple)
    write_dataset("input_bool_nested.txt", nested)
    write_dataset("input_bool_all.txt", simple + nested)

if __name__ == "__main__":
    generate_boolean_datasets()

```

Appendix B - Prompt-Output

(full results in “results.txt” file)

B.1 – [Math GPT]

Sample Predictions (Full)					
<Baseline Model>		<Optimised Model>		<Addition>	
35+99=147	(target: 134)	3+6=13	(target: 9)	90+75=165	(target: 165)
79+67=117	(target: 146)	72+48=120	(target: 120)	36+58=94	(target: 94)
97+24=115	(target: 121)	29+11=40	(target: 40)	32+5=47	(target: 37)
63+64=112	(target: 127)	16+7=114	(target: 23)	71+82=153	(target: 153)
57+98=150	(target: 155)	88+57=154	(target: 145)	65+9=74	(target: 74)
34+95=136	(target: 129)	86+52=138	(target: 138)	64+92=156	(target: 156)
33+96=126	(target: 129)	13+83=95	(target: 96)	15+81=96	(target: 96)
63+80=161	(target: 143)	30+60=80	(target: 90)	38+79=117	(target: 117)
20+45=110	(target: 65)	20+17=47	(target: 37)	83+7=71	(target: 90)
42+32=87	(target: 74)	26+52=77	(target: 78)	27+5=107	(target: 32)
<Subtraction>		<Multiplication>		<Division>	
40-37=3	(target: 3)	14*16=224	(target: 224)	10/2=5	(target: 5)
72-23=49	(target: 49)	19*11=199	(target: 209)	51/1=51	(target: 51)
73-41=32	(target: 32)	13*13=169	(target: 169)	30/30=1	(target: 1)
75-72=3	(target: 3)	17*2=34	(target: 34)	8/4=22	(target: 2)
39-26=13	(target: 13)	9*8=8	(target: 72)	30/15=2	(target: 2)
41-30=11	(target: 11)	10*7=70	(target: 70)	75/75=1	(target: 1)
14-1=23	(target: 13)	2*10=20	(target: 20)	56/1=56	(target: 56)
64-13=51	(target: 51)	18*11=198	(target: 198)	31/31=1	(target: 1)
52-35=17	(target: 17)	0*18=0	(target: 0)	25/1=55	(target: 25)
58-52=6	(target: 6)	12*15=180	(target: 180)	80/80=1	(target: 1)
<Combination>					
40/2=30	(target: 20)				
83/83=1	(target: 1)				
75-39=32	(target: 36)				
59-16=25	(target: 43)				
5+53=121	(target: 58)				
88+60=133	(target: 148)				
47+36=87	(target: 83)				
10/2=3	(target: 5)				
45+87=139	(target: 132)				
88-82=0	(target: 6)				

B.2 – [Boolean Logic GPT]

Sample Predictions (Full)			
<Baseline Model>		< Simple/Optimised Model>	
True AND True=True	(target: True)	False OR True=True	(target: True)
True AND False=False	(target: False)	False OR False=False	(target: False)
False AND False=False	(target: False)	True AND False=False	(target: False)
False XOR False=True	(target: False)	True XOR True=False	(target: False)
True OR False=False	(target: True)	True OR False=True	(target: True)
NOT True=False	(target: False)	NOT False=True	(target: True)
False AND True=False	(target: False)	True OR True=True	(target: True)
False XOR True=True	(target: True)	True AND True=True	(target: True)
NOT False=True	(target: True)	False AND True=False	(target: False)
True OR True=True	(target: True)	True XOR False=True	(target: True)
<Nested>		<Combination>	
(False OR False) AND True=False	(target: False)	False AND False=False	(target: False)
(True XOR True) AND True=False	(target: False)	(True OR False) AND False=False	(target: False)
(False OR False) XOR True=False	(target: True)	(False XOR True) AND True=True	(target: True)
(False XOR True) OR True=True	(target: True)	True XOR False=True	(target: True)
(True XOR True) OR True=True	(target: True)	False XOR False=True	(target: False)
(False AND False) AND True=True	(target: False)	(True AND True) XOR False=True	(target: True)
(False OR True) XOR True=True	(target: False)	(True XOR False) AND True=False	(target: True)
(True XOR False) OR False=False	(target: True)	True AND False=False	(target: False)
(False AND True) OR True=True	(target: True)	True AND True=True	(target: True)
(True XOR False) XOR True=False	(target: False)	(False AND False) AND True=False	(target: False)